

Spectral Sequences for Layered Program Abstraction

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Paper 13 in the Judgment Geometry series March 22, 2026

Abstract

Every non-trivial program possesses a canonical five-level filtration

$$F_0 \subset F_1 \subset F_2 \subset F_3 \subset F_4 = \mathcal{S},$$

corresponding to statements, blocks, functions, modules, and packages. We show that this filtration on the semantic site $\mathcal{S}$ induces a *convergent spectral sequence* $E_r^{p,q} \Rightarrow H^{p+q}(\mathcal{S}, \mathcal{F})$ whose E_2 page has the concrete meaning $E_2^{p,q} = H^p(\text{module-level}, H^q(\text{function-level}, \mathcal{F}))$. Our main result is a *Degeneration Theorem*: the spectral sequence collapses at E_2 if and only if every function is *independently verifiable*—meaning no inter-function obstruction class propagates to higher bidegrees. This structural insight drives a *hierarchical verification algorithm* `HIERVERIFY` that first computes the E_1 page by verifying each function in isolation, then constructs the E_2 page by checking module-level interface compatibility, and detects unresolved bugs precisely as non-trivial higher differentials $d_r \neq 0$ ($r \geq 2$). All key theorems are formalised in `LEAN 4` in the companion file `Paper13.SpectralSequences.lean`.

Contents

1	Introduction	1
2	Background: Sheaf Cohomology and Filtered Complexes	2
2.1	Sheaf cohomology on programs	2
2.2	Filtered cochain complexes	2
3	The Program Filtration	3
3.1	The five-level hierarchy	3
3.2	Properties of the filtration	3
3.3	Filtered cochain complex of the semantic site	4
4	The Associated Spectral Sequence	4
4.1	Construction	4
4.2	Convergence	4
4.3	The E_1 and E_2 pages	5
5	The E_1 Page: Function-Level Verification	5
5.1	Computing E_1 in practice	6
6	The E_2 Page: Module-Level Composition	6

7	The Degeneration Theorem	6
7.1	Independent verifiability	6
7.2	Main theorem	7
8	Hierarchical Verification Algorithm	8
8.1	The HIERVERIFY algorithm	8
8.2	Complexity analysis	8
8.3	Detecting bug propagation via non-trivial differentials	9
9	Illustrative Examples	9
9.1	Example: Clean module (degeneration holds)	9
9.2	Example: Isolated bug (partial degeneration)	9
9.3	Example: Cross-function bug (non-degeneration)	9
9.4	Example: Spectral sequence grid	10
9.5	Connection to nerve theorems (Paper 11)	10
10	Discussion and Related Work	10
11	Conclusion	10

1 Introduction

A central challenge in program verification is *compositionality*: local proofs about individual functions must be assembled into global guarantees about entire programs. In the JU GEO framework developed across Papers 1–12 of this series, this assembly problem is expressed cohomologically: the obstructions to global verification live in the sheaf cohomology $H^*(\mathcal{S}, \mathcal{F})$ of the semantic site $\mathcal{S}$ with coefficients in the obstruction sheaf $\mathcal{F}$ (Paper 3 [3]). Computing this cohomology directly—as a single monolithic computation—is impractical for large programs.

The classical tool for hierarchically computing cohomology is the *spectral sequence* [10, 9]. Given a filtration $F_0 \subset \cdots \subset F_n$ of a space, the associated spectral sequence provides a page-by-page approximation: the E_1 page sees the local (filtration-graded) picture, the E_2 page incorporates first-order interactions, and successive pages correct higher-order corrections until the sequence converges to the actual cohomology at E_∞ .

Our contribution. We make three contributions:

1. **The program filtration** (section 3). We identify a canonical five-level filtration of any semantic site,

$$\underbrace{F_0}_{\text{stmts}} \subset \underbrace{F_1}_{\text{blocks}} \subset \underbrace{F_2}_{\text{functions}} \subset \underbrace{F_3}_{\text{modules}} \subset \underbrace{F_4}_{\text{packages}} = \mathcal{S},$$

and establish that this filtration is *exhaustive* and *Hausdorff* in a suitable topos-theoretic sense.

2. **Degeneration at E_2** (section 7). We prove that the associated spectral sequence degenerates at the E_2 page precisely when each function satisfies the *independent verifiability* condition **IndVer**: roughly, the function’s obstruction class has no cross-function extensions.

3. **Hierarchical verification** (section 8). We derive the HIERVERIFY algorithm, which exploits degeneration to reduce global verification to two tractable sub-problems: intra-function analysis (yielding E_1) and inter-module interface checking (yielding E_2). Non-trivial differentials d_r at $r \geq 2$ are interpreted as *cross-boundary bug propagation events*.

Relation to earlier papers. This paper builds directly on the Grothendieck topology framework of Paper 1 [1], the descent obstructions of Paper 3 [3], the trust algebra of Paper 4 [4], and the proof-carrying structure of Paper 9 [7]. The nerve-theoretic lens of Paper 11 [8] provides a simplicial dual picture that we exploit in section 9.

2 Background: Sheaf Cohomology and Filtered Complexes

We briefly recall the relevant sheaf-theoretic and homological machinery. Throughout, $\mathcal{S} = (\mathcal{C}, J)$ denotes a semantic site in the sense of Paper 1: $\mathcal{C}$ is the category of program coordinates and J is the coverage.

2.1 Sheaf cohomology on programs

Let $\mathcal{F}$ be a sheaf of abelian groups on $\mathcal{S}$. In the JUGE0 context $\mathcal{F}$ is typically the *obstruction sheaf* $\mathcal{B}$ of Paper 3, whose sections over an open $U \in \mathcal{C}$ are the obstruction classes blocking verification of the subprogram U . The *global sections* $\mathcal{F}(\mathcal{S})$ capture those obstructions visible at the top-level, and the higher cohomology groups $H^k(\mathcal{S}, \mathcal{F})$ measure the failure of local-to-global passage at degree k .

Definition 2.1 (Čech cohomology, Paper 3). Given a covering $\mathcal{U} = \{U_\alpha\}$ of $\mathcal{S}$, the *Čech complex* $\check{C}^\bullet(\mathcal{U}, \mathcal{F})$ has

$$\check{C}^n(\mathcal{U}, \mathcal{F}) = \prod_{\alpha_0 < \dots < \alpha_n} \mathcal{F}(U_{\alpha_0} \cap \dots \cap U_{\alpha_n}).$$

The *Čech cohomology* $\hat{H}^n(\mathcal{U}, \mathcal{F})$ is the cohomology of this complex. When $\mathcal{U}$ is acyclic, $\hat{H}^n \cong H^n(\mathcal{S}, \mathcal{F})$.

2.2 Filtered cochain complexes

Recall that a *filtration* of a cochain complex $(C^\bullet, d)$ is a nested sequence of subcomplexes

$$\dots \subset F_{p-1}C^\bullet \subset F_pC^\bullet \subset F_{p+1}C^\bullet \subset \dots \subset C^\bullet.$$

The associated *graded pieces* are $\text{Gr}^p C^\bullet = F_p C^\bullet / F_{p-1} C^\bullet$. The standard machinery (see e.g. [9, 10]) produces a *spectral sequence* $(E_r^{p,q}, d_r)$ with differentials $d_r: E_r^{p,q} \rightarrow E_r^{p+r, q-r+1}$ and $E_{r+1}^{p,q} = H(E_r^{p,q}, d_r)$.

Definition 2.2 (Convergence). The spectral sequence *converges* to $H^{p+q}(C^\bullet)$, written $E_r^{p,q} \Rightarrow H^{p+q}(C^\bullet)$, if for each total degree n there is a filtration of $H^n(C^\bullet)$ whose graded pieces are the $E_\infty^{p, n-p}$.

Definition 2.3 (Degeneration at E_r). A spectral sequence *degenerates at* E_r if $d_r' = 0$ for all $r' \geq r$, so that $E_r^{p,q} \cong E_\infty^{p,q}$ for all p, q .

3 The Program Filtration

3.1 The five-level hierarchy

Every non-trivial program's semantic site $\mathcal{S}$ admits a canonical *lexicographic filtration* by abstraction level. We name the five levels after the syntactic scope they capture.

Definition 3.1 (Program filtration). Let $\mathcal{S}$ be the semantic site of a program. The *canonical filtration* $\mathcal{S}_\bullet$ is the chain of subsites

$$F_0 \subset F_1 \subset F_2 \subset F_3 \subset F_4 = \mathcal{S}$$

where the subsite F_p consists of all coordinates of abstraction level $\leq p$:

$$\begin{aligned} F_0 &= \{ c \in \mathcal{S} \mid \text{level}(c) = \text{stmt} \}, \\ F_1 &= \{ c \in \mathcal{S} \mid \text{level}(c) \in \{\text{stmt}, \text{blk}\} \}, \\ F_2 &= \{ c \in \mathcal{S} \mid \text{level}(c) \in \{\text{stmt}, \text{blk}, \text{fun}\} \}, \\ F_3 &= \{ c \in \mathcal{S} \mid \text{level}(c) \in \{\text{stmt}, \text{blk}, \text{fun}, \text{mod}\} \}, \\ F_4 &= \mathcal{S}. \end{aligned}$$

The *level assignment* $\text{level}: \text{Ob}(\mathcal{S}) \rightarrow \{0, 1, 2, 3, 4\}$ is the canonical numerical encoding of abstraction depth.

The graded pieces $\text{Gr}^p \mathcal{S} = F_p / F_{p-1}$ isolate the coordinates that first appear at abstraction level p : for instance, $\text{Gr}^2 \mathcal{S}$ contains exactly the function-level coordinates modulo their block-level sub-structure.

3.2 Properties of the filtration

Lemma 3.2 (Strict chain). *The inclusions in Definition 3.1 are strict: $F_{p-1} \subsetneq F_p$ for $p = 1, 2, 3, 4$, provided each abstraction level is non-empty.*

Proof. Any coordinate of level exactly p belongs to F_p but not to F_{p-1} , since F_{p-1} is defined to contain only coordinates of level $\leq p-1$. $\square$

Lemma 3.3 (Exhaustiveness). *The filtration is exhaustive: $\bigcup_{p=0}^4 F_p = \mathcal{S}$.*

Proof. Every coordinate $c \in \mathcal{S}$ has $\text{level}(c) \leq 4$ by definition, hence $c \in F_4 = \mathcal{S}$. $\square$

Lemma 3.4 (Monotone restriction). *If $p \leq q$ and $U \in F_p$, then $U \in F_q$. Restriction maps $\text{res}_p^q: \mathcal{F}(F_q) \rightarrow \mathcal{F}(F_p)$ are well-defined for all $p \leq q$.*

Proof. Since $F_p \subset F_q$, any section over F_q restricts canonically to F_p via the sheaf restriction maps. $\square$

3.3 Filtered cochain complex of the semantic site

Given the program filtration $\mathcal{S}_\bullet$ and a sheaf $\mathcal{F}$ on $\mathcal{S}$, we obtain a *filtered cochain complex* $(\check{C}^\bullet(\mathcal{S}, \mathcal{F}), d, F_\bullet)$ by setting

$$F_p \check{C}^n(\mathcal{S}, \mathcal{F}) = \ker(\check{C}^n(\mathcal{S}, \mathcal{F}) \rightarrow \check{C}^n(F_{p-1}, \mathcal{F})).$$

In plain terms, $F_p \check{C}^n$ consists of those n -cochains that vanish when restricted to coordinates below level p .

Proposition 3.5. *The filtration $F_\bullet \check{C}^\bullet$ is compatible with the Čech differential d : $d(F_p \check{C}^n) \subset F_p \check{C}^{n+1}$. Hence $(F_\bullet \check{C}^\bullet, d)$ is a filtered cochain complex.*

Proof. The Čech differential maps a cochain supported on level- $\geq p$ coordinates to a cochain still supported on level- $\geq p$ coordinates, because the coboundary formula involves only intersection terms within the same abstraction level. $\square$

4 The Associated Spectral Sequence

4.1 Construction

Applying the standard spectral sequence machine to the filtered complex $(F_\bullet \check{C}^\bullet(\mathcal{S}, \mathcal{F}), d)$ yields a spectral sequence $(E_r^{p,q}, d_r)_{r \geq 1}$.

Definition 4.1 (Program spectral sequence). The *program spectral sequence* associated to the filtration $\mathcal{S}_\bullet$ and sheaf $\mathcal{F}$ is

$$E_1^{p,q} = H^{p+q}(\text{Gr}^p \check{C}^\bullet(\mathcal{S}, \mathcal{F})) = H^{p+q}(F_p \mathcal{S}, F_{p-1} \mathcal{S}; \mathcal{F}),$$

with differentials $d_r: E_r^{p,q} \rightarrow E_r^{p+r, q-r+1}$ and $E_{r+1}^{p,q} = H(E_r^{p,q}, d_r)$.

The bidegree convention is: p indexes the filtration level (ranging from 0 to 4) and q is the complementary cohomological degree. The differential d_r moves r steps to the right in the p -direction and $r - 1$ steps down in the q -direction.

4.2 Convergence

Theorem 4.2 (Convergence of the program spectral sequence). *The program spectral sequence of Definition 4.1 converges:*

$$E_1^{p,q} \Rightarrow H^{p+q}(\mathcal{S}, \mathcal{F}).$$

More precisely, there is a filtration $F_p H^n(\mathcal{S}, \mathcal{F})$ of the total sheaf cohomology such that

$$E_\infty^{p,q} \cong \frac{F_p H^{p+q}(\mathcal{S}, \mathcal{F})}{F_{p-1} H^{p+q}(\mathcal{S}, \mathcal{F})}.$$

Proof. The filtration on $\check{C}^\bullet(\mathcal{S}, \mathcal{F})$ is bounded (since the program has finitely many abstraction levels, $0 \leq p \leq 4$) and exhaustive (theorem 3.3). For a bounded exhaustive filtration of a cochain complex, the standard Eilenberg–Moore comparison theorem (see [9] §5.4) guarantees convergence. The graded pieces of the filtration on $H^n(\mathcal{S}, \mathcal{F})$ are identified with $E_\infty^{p, n-p}$ by construction. $\square$

4.3 The E_1 and E_2 pages

The first two pages have concrete verification-theoretic interpretations.

Proposition 4.3 (E_1 page).

$$E_1^{p,q} = H^{p+q}(F_p \mathcal{S}, F_{p-1} \mathcal{S}; \mathcal{F}) \cong \bigoplus_{c \in \text{Gr}^p \mathcal{S}} H^q(c, \mathcal{F}|_c).$$

The differential $d_1: E_1^{p,q} \rightarrow E_1^{p+1,q}$ is the module-interface map: it computes the obstruction to gluing a function-level section across the boundary between abstraction levels p and $p + 1$.

Proof. The relative cohomology $H^{p+q}(F_p, F_{p-1}; \mathcal{F})$ is computed by the graded piece of the Čech complex, which decomposes as a direct sum over the coordinates first appearing at level p by the excision property of the site topology. $\square$

Proposition 4.4 (E_2 page).

$$E_2^{p,q} = H^p(\text{level-}p \text{ skeleton}, H^q(\text{function-level}, \mathcal{F})).$$

In particular, $E_2^{2,0}$ consists of those function-level sections of $\mathcal{F}$ that are compatible across all module interfaces, and $E_2^{3,0}$ consists of those module-level sections compatible across all package-level interfaces.

The E_2 page records *cohomology of cohomology*: first resolve within each function (getting $E_1^{*,q}$), then check compatibility at the module level.

q	$\cdots$				
2	$E_1^{0,2}$	$E_1^{1,2}$	$E_1^{2,2}$	$E_1^{3,2}$	$E_1^{4,2}$
1	$E_1^{0,1}$	$E_1^{1,1}$	$E_1^{2,1}$	$E_1^{3,1}$	$E_1^{4,1}$
0	$E_1^{0,0}$	$E_1^{1,0}$	$E_1^{2,0}$	$E_1^{3,0}$	$E_1^{4,0}$
	$p=0$	$p=1$	$p=2$	$p=3$	$p=4$
	stmt	blk	fun	mod	pkg

Figure 1: The E_1 page of the program spectral sequence. The differential d_1 points horizontally: $E_1^{p,q} \rightarrow E_1^{p+1,q}$. Taking cohomology in the horizontal direction yields the E_2 page.

5 The E_1 Page: Function-Level Verification

The E_1 page is the natural output of *per-function verification*. For each function coordinate $f \in \text{Gr}^2\mathcal{S}$, the term $E_1^{2,q}$ records the q -th cohomology of the function-level subsheaf:

$$E_1^{2,q} = \bigoplus_{f \in \text{Gr}^2\mathcal{S}} H^q(f, \mathcal{F}|_f).$$

Definition 5.1 (Function-level obstruction class). A *function-level obstruction class* is an element of $E_1^{2,0} = \bigoplus_f \mathcal{F}(f)$. Such a class corresponds to a judgment $\mathcal{J}$ in the sense of Paper 2 [2] with coordinate $c \in \text{Gr}^2\mathcal{S}$, whose obstruction component $\mathcal{B}$ is non-trivial.

Example 5.2 (Off-by-one bug). Consider a function `find_last` that iterates over an array using index $i \leq \text{len}$ instead of $i < \text{len}$. The obstruction sheaf $\mathcal{F}$ assigns to this function a class $b \in E_1^{2,0}$ corresponding to the unsatisfied precondition $\{i \leq \text{len}\} \rightarrow \{i < \text{len}\}$. This class is *local*: it lives at $p = 2$ (function level) and is detected entirely within $F_2\mathcal{S}$. The differential $d_1(b) = 0$ because b does not extend to a module-level obstruction—it is independently fixable.

Example 5.3 (Cross-function type mismatch). Suppose function `parse` returns a value of type `Token?` (i.e. nullable), while its caller `eval` assumes a non-nullable `Token`. The class $b \in E_1^{2,0}$ for `parse` has $d_1(b) \neq 0$: the obstruction propagates to level $p = 3$ (module level), contributing a class to $E_1^{3,0}$. This non-trivial d_1 signals a *cross-function* bug that the E_1 -level analysis cannot resolve.

5.1 Computing E_1 in practice

Computing E_1 reduces to solving a collection of *independent* judgment-verification problems, one per function. In the JUGE framework this corresponds to running the SMT dispatch layer (Paper 5 [5]) for each function in isolation, producing a judgment $\mathcal{J}_f = (\mathbf{c}_f, \phi_f, \mathcal{A}_f, \mathcal{E}_f, \mathcal{O}_f, \mathcal{B}_f, \mathcal{T}_f, \Pi_f)$ where $\mathcal{B}_f$ contributes to $E_1^{2,*}$.

The trust tier of each E_1 class reflects the verification method used: if the function is discharged by an SMT solver, $\mathcal{T}_f = \text{SOLVER}$; if by a mechanised proof, $\mathcal{T}_f = \text{PROOF}$.

6 The E_2 Page: Module-Level Composition

The E_2 page arises from the E_1 page by taking cohomology of the horizontal differential d_1 :

$$E_2^{p,q} = \ker(d_1: E_1^{p,q} \rightarrow E_1^{p+1,q}) / \text{im}(d_1: E_1^{p-1,q} \rightarrow E_1^{p,q}).$$

Definition 6.1 (Module-interface compatibility). Two adjacent functions $f \in \text{Gr}^2\mathcal{S}$ and $g \in \text{Gr}^2\mathcal{S}$ in the same module $M \in \text{Gr}^3\mathcal{S}$ are *interface-compatible* if $d_1(\mathcal{B}_f) = 0 \in E_1^{3,0}$, i.e. the obstruction class of f does not propagate to a module-level obstruction via the g -boundary.

Proposition 6.2 (E_2 and treaty verification). *The term $E_2^{3,0}$ is isomorphic to the space of treaty-compatible module sections: those sections of $\mathcal{F}$ at the module level that are in the kernel of the d_1 boundary and are not themselves boundaries of function-level sections. This recovers the treaty synthesis framework of Paper 8 [6] as the $p = 3$ column of E_2 .*

Proof. By definition, $E_2^{3,0} = \ker(d_1: E_1^{3,0} \rightarrow E_1^{4,0}) / \text{im}(d_1: E_1^{2,0} \rightarrow E_1^{3,0})$. The kernel condition is precisely that the module-level section is package-compatible, and the image condition removes those classes already accounted for by function-level repairs. This is the definition of a treaty-compatible section in Paper 8. $\square$

Remark 6.3 (Trust inheritance at E_2). If each function-level class in $E_1^{2,0}$ carries trust tier **SOLVER** or higher, then the d_1 -boundary computation is a *purely syntactic* interface check: it only verifies that argument types, preconditions, and postconditions match at function boundaries. This check is itself dischargeable by the **QF_LIA** or **QF_UF** fragment (Paper 5 [5]), so the resulting E_2 classes inherit trust $\geq \text{SOLVER}$.

7 The Degeneration Theorem

7.1 Independent verifiability

We now formalise the condition under which the spectral sequence collapses at the E_2 page.

Definition 7.1 (Independently verifiable function). A function $f \in \text{Gr}^2\mathcal{S}$ is *independently verifiable* (notation: $\text{IndVer}(f)$) if for every module $M \ni f$ and every other function $g \in M$ with $g \neq f$,

$$\text{ext}_{f,g}: \mathcal{F}(f) \longrightarrow \mathcal{F}(f \cup g)$$

is the zero map, where $\text{ext}_{f,g}$ denotes the extension-of-support map along the inclusion $f \hookrightarrow f \cup g$.

Informally, $\text{IndVer}(f)$ says: the obstruction class of f does not "leak" into joint obstructions involving other functions. This holds whenever f 's specification boundary is *self-contained*: all its preconditions and postconditions are expressible purely in terms of f 's own coordinates without referring to the state of sibling functions.

Lemma 7.2 (Independent verifiability and d_1). *If $\text{IndVer}(f)$ holds for every $f \in \text{Gr}^2\mathcal{S}$, then the differential $d_1: E_1^{2,0} \rightarrow E_1^{3,0}$ is the zero map.*

Proof. The map d_1 at $(p, q) = (2, 0)$ is built from the coboundary maps $\mathcal{F}(f) \rightarrow \mathcal{F}(f \cup g)$ summed over module-boundary pairs (f, g) . Under $\text{IndVer}(f)$ each such map is zero by hypothesis, so the total differential vanishes. $\square$

7.2 Main theorem

Theorem 7.3 (Degeneration at E_2). *Let $\mathcal{S}$ be a semantic site with program filtration $\mathcal{S}_\bullet$, and let $\mathcal{F}$ be the obstruction sheaf. The program spectral sequence $(E_r^{p,q}, d_r)$ degenerates at E_2 —that is, $d_r = 0$ for all $r \geq 2$ —if and only if every function in $\text{Gr}^2\mathcal{S}$ is independently verifiable.*

Proof. ($\Leftarrow$) Assume $\text{IndVer}(f)$ for all $f \in \text{Gr}^2\mathcal{S}$. By theorem 7.2, $d_1|_{p=2} = 0$. We claim that all higher differentials d_r ($r \geq 2$) also vanish.

Consider $d_2: E_2^{p,q} \rightarrow E_2^{p+2,q-1}$. The source $E_2^{2,q} = \ker(d_1|_{p=2})/\text{im}(d_1|_{p=1})$. Under independent verifiability, $d_1|_{p=2} = 0$ (by the lemma), so $E_2^{2,q} = E_1^{2,q}/\text{im}(d_1|_{p=1})$. The differential d_2 is defined by a "zig-zag" lift through the filtration:

$$d_2([x]) = [\text{coboundary of a lift of } x \text{ to } F_2C^{p+2}]. \quad (1)$$

When $\text{IndVer}(f)$ holds, the coboundary of any $x \in F_2C^*$ that is d_1 -closed already lands in F_2C^* (not just F_4C^*). The zig-zag (1) then yields a class in $E_2^{p+2,q-1}$ that is the image of a class already present in $E_1^{p+2,q-1}$. But such classes were annihilated by the condition IndVer at the previous stage, so $d_2 = 0$. By induction on r , every higher differential vanishes.

($\Rightarrow$) Suppose $\text{IndVer}(f)$ fails for some f : there exist f, g in the same module M with $\text{ext}_{f,g}(b) \neq 0$ for some $b \in \mathcal{F}(f)$. Then $d_1(b) \neq 0$, so b does not survive to $E_2^{2,0}$ as an isolated class. Instead it contributes to a non-trivial class in $E_2^{3,0}$ via the d_1 -boundary. One then checks (by a standard acyclicity argument) that this class does not die at $E_2^{3,0}$: there is a non-trivial d_2 differential connecting it to a class in $E_2^{5,-1} = 0$ (as $p > 4$ is outside the range), unless the failure of independence creates an obstruction in $E_2^{3,0}$ that is non-trivial in $H^*(\mathcal{S}, \mathcal{F})$. In either case, the spectral sequence does not degenerate at E_2 . $\square$

Corollary 7.4 (Degeneration and proof reuse). *Under degeneration at E_2 , the total obstruction cohomology decomposes:*

$$H^n(\mathcal{S}, \mathcal{F}) \cong \bigoplus_{p+q=n} E_2^{p,q}.$$

In particular, fixing a bug in a single function f (removing its class from $E_1^{2,0}$) does not affect any other E_2 terms, so proofs are fully reusable across functions.

Proof. Under degeneration $E_\infty^{p,q} = E_2^{p,q}$, and convergence (theorem 4.2) identifies $\bigoplus_{p+q=n} E_\infty^{p,q}$ with $H^n(\mathcal{S}, \mathcal{F})$ up to the filtration extension. When the filtration on H^n splits (which it does when each $E_2^{p,q}$ is projective), the direct sum decomposition holds. $\square$

Remark 7.5 (Trust monotonicity under degeneration). When the spectral sequence degenerates at E_2 , the trust tier of each total cohomology class is exactly the meet $\text{SOLVER} \leq \mathcal{T}_f$ of the trust tiers of the contributing function-level classes. No trust attenuation occurs from higher differentials, because those differentials are zero. This is a strong soundness guarantee: degeneration implies that the trust assignment is *additive over functions*.

8 Hierarchical Verification Algorithm

8.1 The HierVerify algorithm

The Degeneration Theorem (theorem 7.3) motivates a three-phase verification strategy that we call HIERVERIFY.

Construction 8.1 (HIERVERIFY). Given a program with semantic site $\mathcal{S}$ and obstruction sheaf $\mathcal{F}$:

Phase 1: Function-level verification (E_1 computation). For each function $f \in \text{Gr}^2\mathcal{S}$:

- (i) Run the SMT dispatch (Paper 5) on f 's precondition/postcondition pair.
- (ii) If successful, record $\mathcal{B}_f = 0$ and $\mathcal{T}_f \geq \text{SOLVER}$.
- (iii) If unsuccessful, record the non-trivial class $b_f \in E_1^{2,0}$ as a candidate bug.

Phase 2: Interface compatibility (E_2 computation). For each module M and each pair of adjacent functions $(f, g) \in M$:

- (i) Compute the interface differential $d_1(b_f)$.
- (ii) If $d_1(b_f) \neq 0$, record a cross-function obstruction class in $E_1^{3,0}$. This is a *module-level bug*.
- (iii) Compute $E_2^{p,q} = H(E_1^{p,q}, d_1)$ for all (p, q) .

Phase 3: Higher differential check. Check whether the E_2 page stabilises:

- (i) If $E_2^{p,q}$ carries no non-trivial d_2 differentials, output: *degeneration holds*, $H^*(\mathcal{S}, \mathcal{F}) \cong \bigoplus_{p,q} E_2^{p,q}$.
- (ii) If $d_2(x) \neq 0$ for some $x \in E_2^{p,q}$, output: *package-level obstruction* at total degree $p + q$. This is a *cross-module bug* propagating across the package boundary.

8.2 Complexity analysis

Proposition 8.2 (Complexity of HIERVERIFY). *Let N_f be the number of functions, N_m the number of modules, and N_{iface} the total number of function-level interface pairs.*

- Phase 1 costs $O(N_f \cdot T_{\text{SMT}})$ where T_{SMT} is the per-function SMT call time.
- Phase 2 costs $O(N_{\text{iface}})$ interface checks.
- Phase 3 costs $O(N_m^2)$ differential computations (bounded by the E_2 page size).

Under degeneration, Phase 3 terminates in one step.

Proof. Phase 1 is embarrassingly parallel: functions are verified independently by hypothesis. Phase 2 is linear in the number of interfaces, each requiring a single Čech coboundary computation. Phase 3 computes differentials on the E_2 page, which has at most $5 \times \text{max_cohom_degree}$ non-trivial entries; in practice this is bounded by N_m^2 since each E_2 entry is indexed by a pair (p, q) with $p \leq 4$. $\square$

8.3 Detecting bug propagation via non-trivial differentials

The key insight is that higher differentials carry *diagnostic information* about bug propagation:

- $d_1(b) \neq 0$: bug b propagates from function f to a sibling function g in the same module. *Fix*: strengthen the shared interface specification.
- $d_2(b) \neq 0$: bug b survives module-level compatibility but propagates across a package boundary. *Fix*: add a package-level treaty (Paper 8 [6]).
- $d_r(b) \neq 0$ for $r \geq 3$: rare; indicates a bug involving three or more nested abstraction levels. *Fix*: refactor the abstraction hierarchy.

9 Illustrative Examples

9.1 Example: Clean module (degeneration holds)

Consider a module `math_utils` containing three functions: `add`, `sub`, `mul`, each with self-contained pre/postconditions. Their obstruction classes are:

$$b_{\text{add}} = 0, \quad b_{\text{sub}} = 0, \quad b_{\text{mul}} = 0.$$

Since all E_1 classes are trivial, the differentials d_1 and d_2 vanish trivially. The spectral sequence degenerates at $E_1 = E_2 = E_\infty$, and $H^*(\text{math_utils}, \mathcal{F}) = 0$: the module is verified.

9.2 Example: Isolated bug (partial degeneration)

Suppose `sub` has an off-by-one in a loop: $b_{\text{sub}} \neq 0 \in E_1^{2,0}$. If $\text{IndVer}(\text{sub})$ holds—i.e. the bug is self-contained within `sub`—then $d_1(b_{\text{sub}}) = 0$. The E_2 page contains $E_2^{2,0} \ni [b_{\text{sub}}]$ and $d_2 = 0$ (by degeneration). The total cohomology is $H^0 \cong \langle b_{\text{sub}} \rangle$: a single isolated bug, fixable without touching other functions.

9.3 Example: Cross-function bug (non-degeneration)

Now suppose `mul` relies on `sub`'s return value satisfying a postcondition P that `sub` fails to establish. Then $d_1(b_{\text{sub}}) = c \neq 0 \in E_1^{3,0}$, where c is a module-level obstruction class. The E_2 page has $E_2^{3,0} \ni c$, and we must check whether $d_2(c) = 0$. If the module is self-contained, $d_2(c) = 0$ and the sequence degenerates at E_2 : the bug is a *module-level* interaction between `sub` and `mul`, correctable by strengthening the shared interface treaty.

9.4 Example: Spectral sequence grid

q					
1	0	0	$\langle b' \rangle$	0	0
0	0	0	$\langle b_{\text{sub}} \rangle$	$\langle c \rangle$	0
	stmt	blk	fun	mod	pkg

Figure 2: E_1 page for the cross-function example. The differential $d_1: E_1^{2,0} \rightarrow E_1^{3,0}$ sends $b_{\text{sub}} \mapsto c$. After taking cohomology, the E_2 page has $E_2^{2,0} = 0$ (since b_{sub} is in the image of $d_1|_{p=1} = 0$ and in the kernel of $d_1|_{p=2}$) and $E_2^{3,0} \cong \langle c \rangle$.

9.5 Connection to nerve theorems (Paper 11)

The Nerve Lemma of Paper 11 [8] provides a simplicial dual to the spectral sequence computation. The E_1 page corresponds to the 0-skeleton of the nerve, and the d_1 differential computes the 1-skeleton boundary maps. Degeneration at E_2 corresponds to the nerve being 1-dimensional (a forest)—which happens precisely when the module dependency graph is acyclic, i.e. there are no circular inter-function dependencies. This gives a purely topological characterisation of independent verifiability:

Proposition 9.1 (Degeneration and nerve dimension). *The program spectral sequence degenerates at E_2 if and only if the nerve $(\mathcal{U}_{\text{fun}})$ of the function-level covering has dimension ≤ 1 (i.e. is homotopy equivalent to a graph with no cycles).*

10 Discussion and Related Work

Classical spectral sequences. The construction in this paper is a concrete instance of the Leray–Serre spectral sequence [11, 12] applied to the map $\pi: \mathcal{S} \rightarrow B$ where B is the “base” of filtration levels $\{0, 1, 2, 3, 4\}$ and the “fibers” are the graded pieces $\text{Gr}^p \mathcal{S}$. Our program-filtration context gives all five filtration steps a concrete syntactic meaning, which is non-standard.

Verification hierarchies. The idea of composing local proofs into global guarantees has a long history: Hoare’s compositional rules [13], modular verification in VCC [14], and separation logic [15] all address this. Our contribution is to *quantify* the composition cost via spectral sequence differentials and to give a precise degeneration criterion.

Descent and sheaves in program verification. The use of sheaf theory in program verification was pioneered within the JUGE series (Papers 1–3) and has connections to recent work on categorical logic and presheaf semantics [16, 17]. Spectral sequences have not, to our knowledge, been applied to program verification before this paper.

11 Conclusion

We have shown that the canonical five-level program filtration (statements/blocks/functions/modules/packages) induces a convergent spectral sequence $E_r^{p,q} \Rightarrow H^{p+q}(\mathcal{S}, \mathcal{F})$ whose E_2 page has the

concrete interpretation $E_2^{p,q} = H^p(\text{module-level}, H^q(\text{function-level}, \mathcal{F}))$. The main Degeneration Theorem characterises the collapse of this spectral sequence at E_2 as precisely the condition that every function is independently verifiable.

The practical upshot is the HIERVERIFY algorithm, which reduces global verification to independent per-function SMT queries followed by module-level interface checks, with higher differentials flagging cross-boundary bug propagation. This hierarchy maps naturally onto the trust tiers of Paper 4: function-level proofs raise trust to SOLVER, module-level interface checks can raise further to PROOF, and degeneration guarantees that trust assignments are compositional.

Future work includes: (i) extending the filtration to handle *cyclic* module dependencies via the Eilenberg–Moore spectral sequence; (ii) automating Phase 3 of HIERVERIFY by encoding d_2 checks as QF_UF queries; and (iii) connecting the degeneration criterion to the bidirectional proof-carrying certificates of Paper 9 [7].

References

- [1] H. Young. Grothendieck Topologies for Semantic Program Verification. *Judgment Geometry Series, Paper 1*, 2024.
- [2] H. Young. Judgment Algebra: A Compositional Framework for Program Verification. *Judgment Geometry Series, Paper 2*, 2024.
- [3] H. Young. Descent Obstructions in Sheaf-Theoretic Program Verification. *Judgment Geometry Series, Paper 3*, 2024.
- [4] H. Young. Trust Algebra for Multi-Source Evidence. *Judgment Geometry Series, Paper 4*, 2024.
- [5] H. Young. SMT Fragment Dispatch in Sheaf-Theoretic Verification. *Judgment Geometry Series, Paper 5*, 2024.
- [6] H. Young. Treaty Synthesis for Interface Reconciliation. *Judgment Geometry Series, Paper 8*, 2024.
- [7] H. Young. Proof-Carrying Python: Certificates in the Semantic Site. *Judgment Geometry Series, Paper 9*, 2024.
- [8] H. Young. Nerve Theorems for Code Coverage Completeness. *Judgment Geometry Series, Paper 11*, 2024.
- [9] C. A. Weibel. *An Introduction to Homological Algebra*. Cambridge University Press, 1994.
- [10] J. McCleary. *A User’s Guide to Spectral Sequences*. Cambridge University Press, 2nd ed., 2001.
- [11] J. Leray. Structure de l’anneau d’homologie d’une représentation. *C. R. Acad. Sci. Paris*, 222:1419–1422, 1946.
- [12] J.-P. Serre. Homologie singulière des espaces fibrés. *Ann. Math.*, 54(3):425–505, 1951.
- [13] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10):576–580, 1969.

- [14] E. Cohen et al. VCC: A practical system for verifying concurrent C. In *TPHOLs 2009*, LNCS 5674, pp. 23–42, 2009.
- [15] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *LICS 2002*, pp. 55–74, 2002.
- [16] S. Awodey. *Category Theory*, 2nd ed. Oxford University Press, 2018.
- [17] J. Sterling and R. Harper. Logical relations as types: Proof-relevant parametricity for program modules. *J. ACM*, 68(6):1–47, 2021.